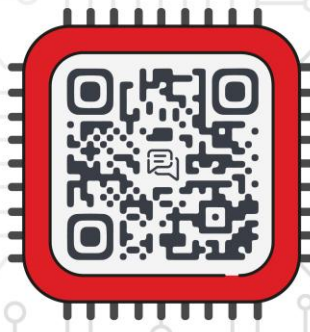




2



14



1680 sp

نظري

كلية الهندسة المعلوماتية

السنة الثانية

Binary Search Trees

د. مدحت الصوص

خوارزميات ٢

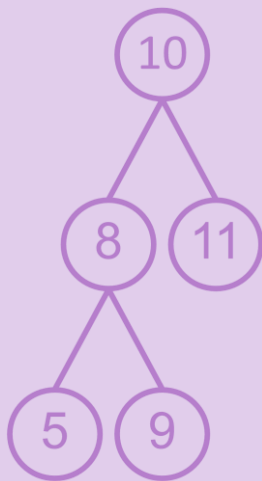
RB Informatics; 5/4/2025

محتوى مجاني غير مخصص للبيع التجاري

Binary Search Tree

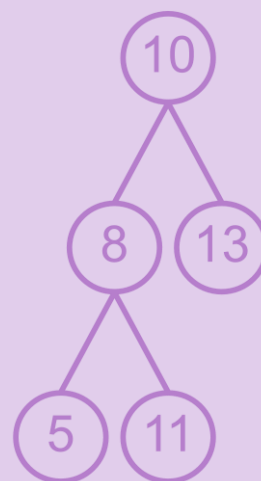
ما هي شجرة البحث الثنائية (BST)؟

هي شجرة ثنائية تساعد في البحث عن عنصر معين، ولكن نضع لها قيد معين على ال Data المخزنة فيها وهو: كل العقد على يمين الجذر تكون أكبر من الجذر، وكل العقد على يسار الجذر تكون أصغر من الجذر. ويطبق نفس الأمر على كل ال subtree الموجودة في الشجرة، مثلاً:



هذه BST

لأن كل العناصر على يمين الجذر أكبر من الجذر، وكل العناصر على يسار الجذر أصغر من الجذر



هذه ليست BST

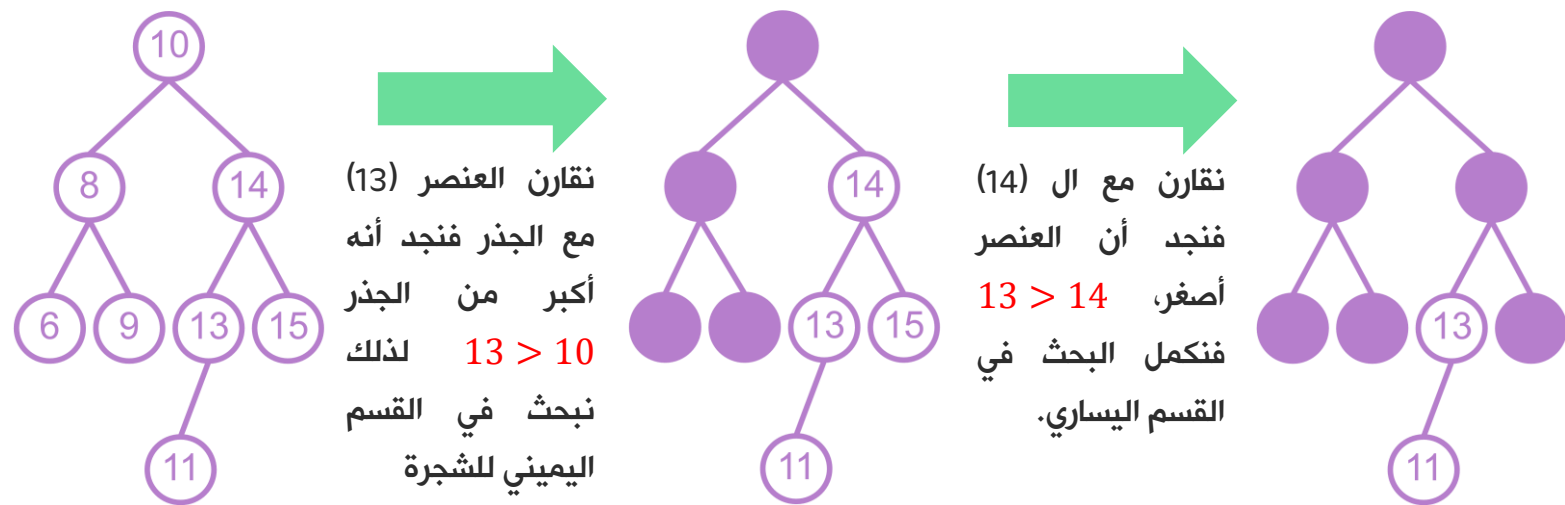
لوجود العقدة 11 على يسار الجذر وهي أكبر من العقدة 10

Search In BST

طريقة البحث في ال (BST):

تكون بمقارنة العنصر الذي نريد إيجاده بالجذر، فإذا كان العنصر أكبر من الجذر نبحث في الجزء اليميني للشجرة ونستغني عن كامل القسم اليساري، أما إذا كان العنصر أصغر من الجذر نبحث في الجزء اليساري للشجرة ونستغني عن كامل القسم اليميني. (ونكمل بنفس الطريقة كل ال SubTree للشجرة).

مثلاً لو كان لدينا الشجرة التالية وكنا نريد البحث عن العنصر (13):



- **نلاحظ** أننا وجدنا العنصر (13) دون اللجوء للبحث في كل عناصر الشجرة.

ملاحظة:

تكلّمنا في المحاضرة السابقة أنه يوجد 3 طرق لتمثيل الأشجار، وهي: pointers - one dimensional array - two dimensional array
لكننا سوف نعتمد في دراستنا على تمثيل الأشجار بطريقة ال pointers لأنها الأكثر انتشاراً.
ولذلك جميع الأكواد التي سوف نذكرها في المحاضرات ستكون على الأشجار الممثلة باستخدام ال pointers.



Implementation

يمكن كتابة الخوارزمية السابقة بطريقتين مختلفتين:

1. باستخدام الحل العودي.

2. باستخدام الحل التكراري.

أولاً: كتابة الخوارزمية باستخدام الحل العودي:

```
class Node {
    int value;
    Node left;
    Node right;

    public Node(int value) {
        this.value = value;
        left = right = null;
    }
}

Node search(Node node, int x) {
    if (node == null)
        return null;
    if (node.value == x)
        return node;
    if (node.value > x)
        return search(node.left, x);
    else
        return search(node.right, x);
}
```

ملاحظة هامة:

عند استدعاء التابع search بشكل عودي
استخدمنا التعليمة return قبل الاستدعاء
لكي نرسل النتيجة إلى الاستدعاء السابق،
وهكذا حتى نصل إلى المكان الذي بدأنا منه
أول استدعاء للتابع، وبذلك يمكننا التأكد أن
كل استدعاء يحصل على النتيجة الصحيحة من
الاستدعاء الفرعي.

شرح الكود:

- نعرف class يحوي على صفات العقدة (Node) التي نريد التخزين فيها، فنخزن مؤشر للابن اليميني ومؤشر للابن اليساري وال Data داخل العقدة.
- بما أننا نريد البحث عن عقدة تحتوي على عدد معين لذلك نعرف تابع search من نوع Node لأننا نريد أن نرجع العقدة التي تحتوي على العنصر.
- إذا وصلنا إلى Node قيمتها تساوي ال null هذا يعني أن العنصر غير موجود في الشجرة، لذلك ننهي استدعاء التابع بإرجاع قيمة null.
- إذا وصلنا إلى العدد الذي نبحث عنه، نرجع ال Node التي تحتوي هذا العدد.
- أما إذا لم نصل إلى العنصر، وما زال يوجد عناصر في الشجرة فإننا نقارن العنصر المراد البحث عنه مع العقدة التي وصلنا إليها، فإذا كان العنصر أكبر من العقدة الحالية، نستدعي تابع ال search بشكل عودي ولكن مع الانتقال إلى ال Node الموجودة على يسار هذه العقدة.
- أما إذا كان العنصر أصغر من العقدة الحالية فنستدعي تابع ال search بشكل عودي ولكن مع الانتقال إلى ال Node الموجودة على يمين هذه العقدة.
- نستمر في الانتقال حتى نصل إلى العنصر الذي نبحث عنه أو حتى نصل إلى نهاية الشجرة دون العثور عليه فيرجع التابع قيمة null.

ثانياً: كتابة الخوارزمية باستخدام الحل التكراري:

```

class Node {
    int value;
    Node left;
    Node right;
    public Node(int value) {
        this.value = value;
        left = right = null;
    }
}

Node search(Node node, int x) {
    while (node != null && node.value != x) {
        if (node.value > x)
            node = node.left;
        else
            node = node.right;
    }
    return node;
}

```

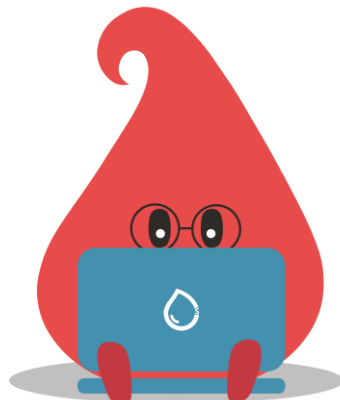
شرح الكود:

- نبدأ بتعريف ال class الخاص بالعقد.
- شرط الخروج من الحلقة إما أن نصل إلى قيمة null أي لم نجد العنصر المطلوب، أو نجد العنصر الذي نبحث عنه (x).
- نقارن العنصر المراد البحث عنه بالعقدة الحالية، مثل الطريقة العودية.
- نستمر داخل الحلقة حتى يختل أحد الشرطين فإما نجد العنصر فنرجع ال Node الخاصة به. أو لا نجده فنرجع null.

ملاحظة:

في لغة java ، عند إرسال كائن (object) إلى دالة يتم إرسال نسخة من عنوان هذا الكائن (object) لذلك إذا قمنا بتغيير العنوان داخل الدالة (function) فهذا لن يؤثر على الكائن الأساسي، أي مثلاً: (node = node.left;) لن يؤثر على الكائن الأساسي خارج ال (function)، أما إذا قمنا بتعديل محتوى الكائن نفسه (مثل node.value = 10) فإن هذا التعديل سوف يظهر على الكائن الأصلي خارج الدالة.

باختصار: إذا قمنا بتغيير العنوان فهذا لن يؤثر لأنه يتم إرسال نسخة من عنوان الكائن. أما إذا قمنا بتعديل القيم داخل الكائن فهذا سوف يؤثر على الكائن الأساسي خارج ال (function) لأن العنوان الأساسي والنسخة من العنوان التي أرسلت ل (function) يشيران إلى نفس الكائن (object).



Time complexity

⚡ تعقيد الخوارزمية:

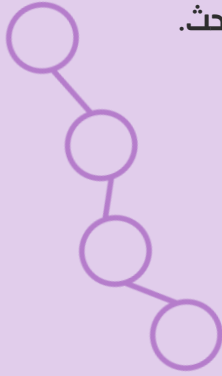
في كلا الحليين نلاحظ أننا نقسم الشجرة إلى نصفين، أي في أول دخول للشجرة نستغني عن نصف الشجرة وفي الدخول الثاني نستغني عن نصف النصف وهكذا حتى نصل إلى العنصر المراد البحث عنه أو إلى نهاية الشجرة.

⚡ ولكن كم مرة سوف نكرر هذه العملية؟

سوف نكرر هذه العملية حتى نصل إلى مكان لا يمكننا الاستغناء عن نصف الشجرة فيه وبالتعريف يكون هذا العدد يساوي $\log_2 n$ وهو يمثل كم مرة يمكننا تقسيم الشجرة إلى قسمين.

ملاحظة:

ليس من الضروري أن يكون دائماً تعقيد البحث هو $O(\log_2 n)$ حيث يعتبر هذا التعقيد هو الحالة الوسطية للبحث، فمثلاً إذا كانت الشجرة خطية فإن تعقيد البحث فيها هو $O(n)$ ، وهو أسوأ حالة للبحث.



- نلاحظ أن في هذه الشجرة سوف نضطر إلى المرور بجميع العقد لكي نصل إلى العنصر المراد البحث عنه أو نصل إلى نهاية الشجرة.

ويمكن أيضاً أن يكون العنصر المراد البحث عنه هو أول عنصر في الشجرة، ويكون التعقيد عندئذ هو $O(1)$ وهو أفضل حالة ممكنة للتعقيد.

ومنه فإن:

أفضل حالة	الحالة الوسطية	أسوأ حالة للبحث
$O(1)$	$O(\log_2 n)$	$O(n)$

⚡ تلخيص:

في شجرة البحث الثنائية يكون التعقيد دائماً هو نفسه ارتفاع الشجرة (H) أي في كل حالات البحث التعقيد هو $O(H)$. فلو كانت الشجرة خطية فإن ارتفاعها هو (n) ، وبالتالي فإننا نحتاج بالمرور بجميع العناصر حتى نصل للعنصر المراد البحث عنه. ويكون التعقيد هو ارتفاع الشجرة. $O(n) = O(H)$.

الإضافة Insertion in BST

يوجد لدينا حالتين في الإضافة:

- إضافة عقدة كورقة (Leaves add): ستكون العقدة المضافة ورقة جديدة (leave) ، أي حصراً بلا أبناء.
- إضافة عقدة كجذر (Root add): ستكون العقدة المضافة الجذر الجديد للشجرة.

أولاً: إضافة عقدة كورقة (Leaves add):

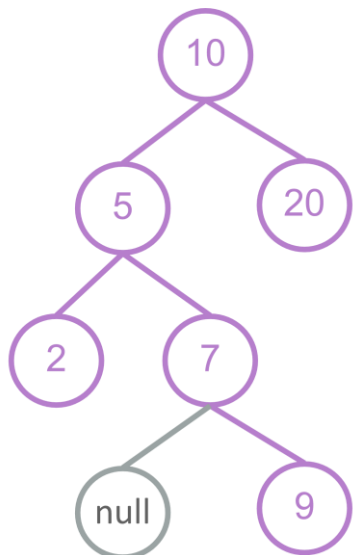
بدايةً يجب تحديد المكان الأنسب للعقدة التي نريد إضافتها بحيث نحافظ على مفهوم شجرة البحث الثنائية باستخدام آلية البحث التي درسناها سابقاً.

آلية التفكير:

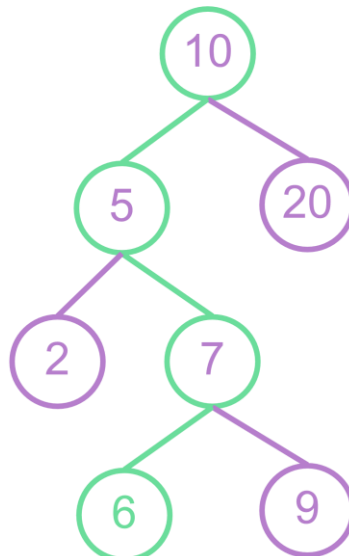
نقارن قيمة العقدة المراد إضافتها مع قيمة الجذر أولاً :

- إذا كانت أكبر نذهب للقسم الأيمن.
 - إذا كانت أصغر نذهب للقسم الأيسر.
- نكرر العملية على كل جذور الأشجار الجزئية (subTrees) حتى الوصول لمكان فارغ، أي حتى نصل ل Null ، وهو المكان الذي سنضيف فيه العنصر الجديد.

مثال: ليكن لدينا الشجرة التالية:



إذا اردنا إضافة الرقم (6) مثلاً، فسوف نقارن مع كل عنصر نمر عليه، فإذا كان أكبر من (6) نكمل في القسم اليساري وإذا كان أصغر من (6) نكمل في القسم اليميني، ونستمر بالانتقال حتى نصل لمكان فارغ (null) ونضيف العقدة الجديدة فيه.



ملاحظة:

تعقيد إضافة ورقة هو نفس تعقيد البحث في الشجرة أي التعقيد يساوي ارتفاع الشجرة $O(H)$.

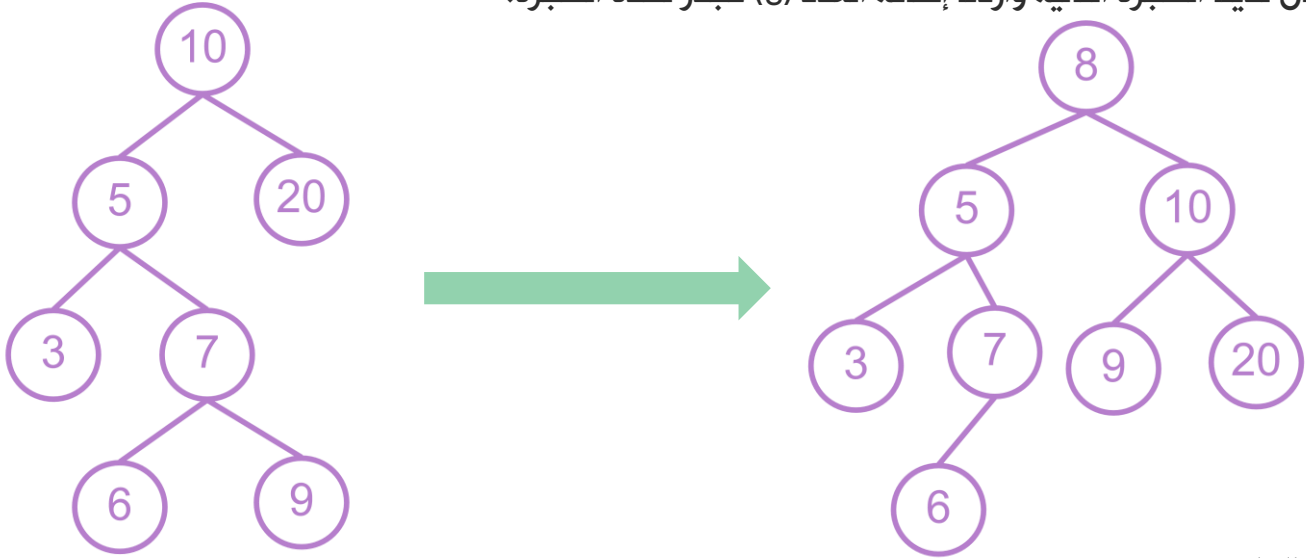
ثانياً: إضافة عقدة كجذر (Root add):

آلية التفكير الأولى:

- نقوم بتفكيك الشجرة وإعادة تشكيلها من البداية، أي نضع الرقم الذي نريد اضافته كجذر ونضيف باقي العناصر بعده عنصراً تلو الآخر وفق مبدأ ال BST.

مثال:

لو كان لدينا الشجرة التالية وأردنا إضافة العدد (8) كجذر لهذه الشجرة:



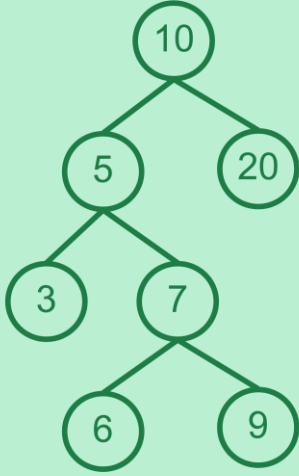
شرح الحل:

- قمنا أولاً بوضع ال (8) كجذر، ثم بدأنا بإضافة باقي العقد واحدة تلو الأخرى وفق مبدأ ال BST. فمثلاً: قمنا بإضافة ال (5) على يسار العقدة (8) ثم أضفنا ال (7) ثم ال (3) ثم ال (10) وهكذا حتى نضيف جميع العقد.
- ملاحظة:** لا يوجد ترتيب محدد لإضافة العناصر الى الشجرة، فمثلاً يمكننا إضافة ال (7) قبل ال (5).
- تعقيد هذا الحل:
- نلاحظ أننا في هذه الطريقة سنضطر إلى إعادة المرور على جميع العناصر بتعقيد $O(n)$ وإعادة ترتيبها في الشجرة الجديدة بتعقيد $O(H)$ لكل عقدة، ولذلك فإن التعقيد يكون $O(n \times H)$ أي عدد العناصر $\times$ ارتفاع الشجرة، ويمكن تحسين هذا التعقيد كما سنلاحظ في طريقة الحل الثانية.

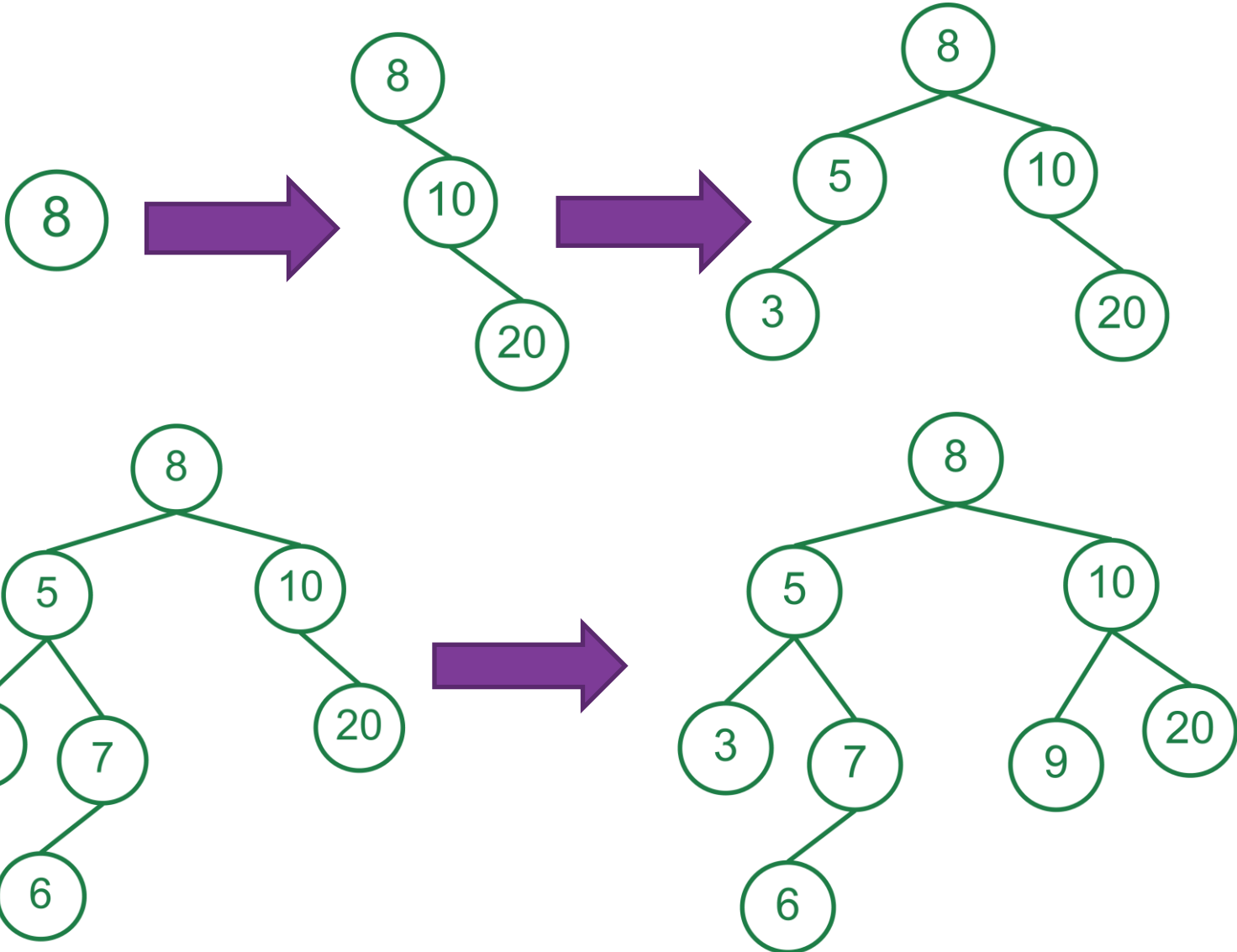
آلية التفكير الثانية:

- نقارن قيمة العقدة المراد إضافتها كجذر جديد للشجرة مع قيمة الجذر السابق :
- إذا كانت أكبر منه ننقل الجذر السابق وشجرته الفرعية من جهة اليسار (أي ال left subtree الخاصة بالجذر السابق) لتصبح ابن أيسر للجذر الجديد.
- إذا كانت أصغر منه ننقل الجذر السابق وشجرته الفرعية من جهة اليمين (أي ال right subTree الخاصة بالجذر السابق) لتصبح ابن يمين للجذر الجديد .
- وهكذا يمكننا إضافة الجذر دون الحاجة للمرور على جميع العناصر ومقارنتها بالجذر الجديد.

لو كان لدينا الشجرة التالية وأردنا إضافة الرقم (8) كجذر لهذه الشجرة دون اللجوء لإعادة تشكيلها:



1. نضع ال (8) كجذر للشجرة.
2. نقارن الجذر الجديد (8) مع الجذر القديم (10) فنجد أن $10 > 8$ لذلك نأخذ ال (10) وجميع العناصر على يمينها ونضعها على يمين ال (8).
3. نقارن ال (5) مع ال (8) فنجد أن $5 < 8$ ، لذلك نأخذ ال (5) وجميع العناصر على يسارها ونضعها على يسار ال (8).
4. الآن نقارن ال (7) مع ال (8) فنجد أن $7 < 8$ ، فنذهب الى جهة اليسار، ثم نقارن ال (7) مع ال (5) فنجد أن $7 > 5$ ، لذلك نأخذ ال (7) وجميع العناصر على يسارها ونضعها على يمين ال (5).
5. أخيرا نقارن ال (9) مع ال (8) فنجد أن $9 > 8$ ، فنذهب إلى جهة اليمين، ثم نقارن ال (9) مع ال (10) فنجد أن $9 < 10$ ، لذلك نضيفها على يسار ال (10).



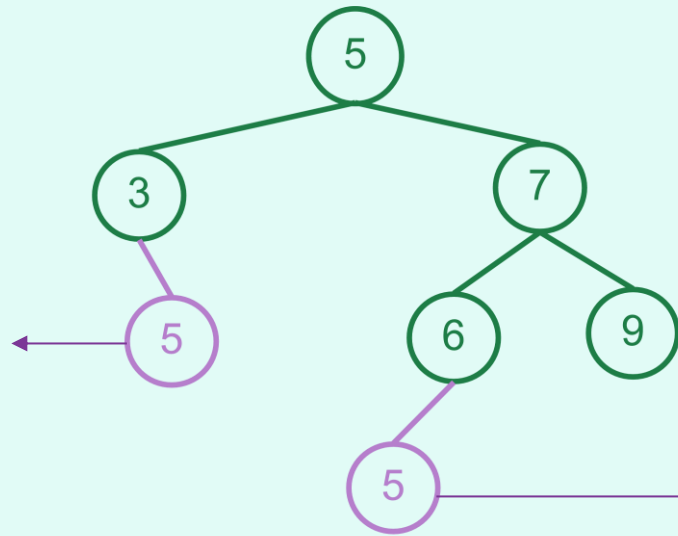
- تعقيد هذا الحل هو من رتبة ارتفاع الشجرة أي $O(H)$.
- نستفيد من إضافة عقدة جديدة للجذر عندما يكون احتمال البحث عن العقدة المضافة مؤخراً أكبر، حيث تصبح عملية البحث أسرع لأنه العنصر الأقرب.

سؤال:

لو أردنا إضافة عنصر كورقة، وكان هذا العنصر موجود مسبقاً في الشجرة:

هناك حلان لهذه المشكلة:

- الحل الأول: هو أن نضع عداد في كل عقدة، فعند تكرار العنصر في الشجرة نفسها يزيد هذا العداد بمقدار التكرار لهذا العنصر.
- الحل الثاني: هو أن نعرف الشجرة بطريقة تسمح بتكرار العناصر، أي سوف نغير بنية الشجرة. بحيث يكون الأبناء اليسار أصغر أو تساوي الجذر والأبناء اليمين أكبر تماماً من الجذر أو يكون الأبناء اليمين أكبر أو تساوي الجذر والأبناء اليسار أصغر تماماً من الجذر (نحدد ذلك على رغبتنا داخل الكود).
- مثال: إذا أردنا إضافة العنصر (5) للشجرة التالية:



سيكون مكان العنصر
هنا إذا كان تعريف
الشجرة أن الأبناء
اليسار أصغر أو تساوي
الجذر.

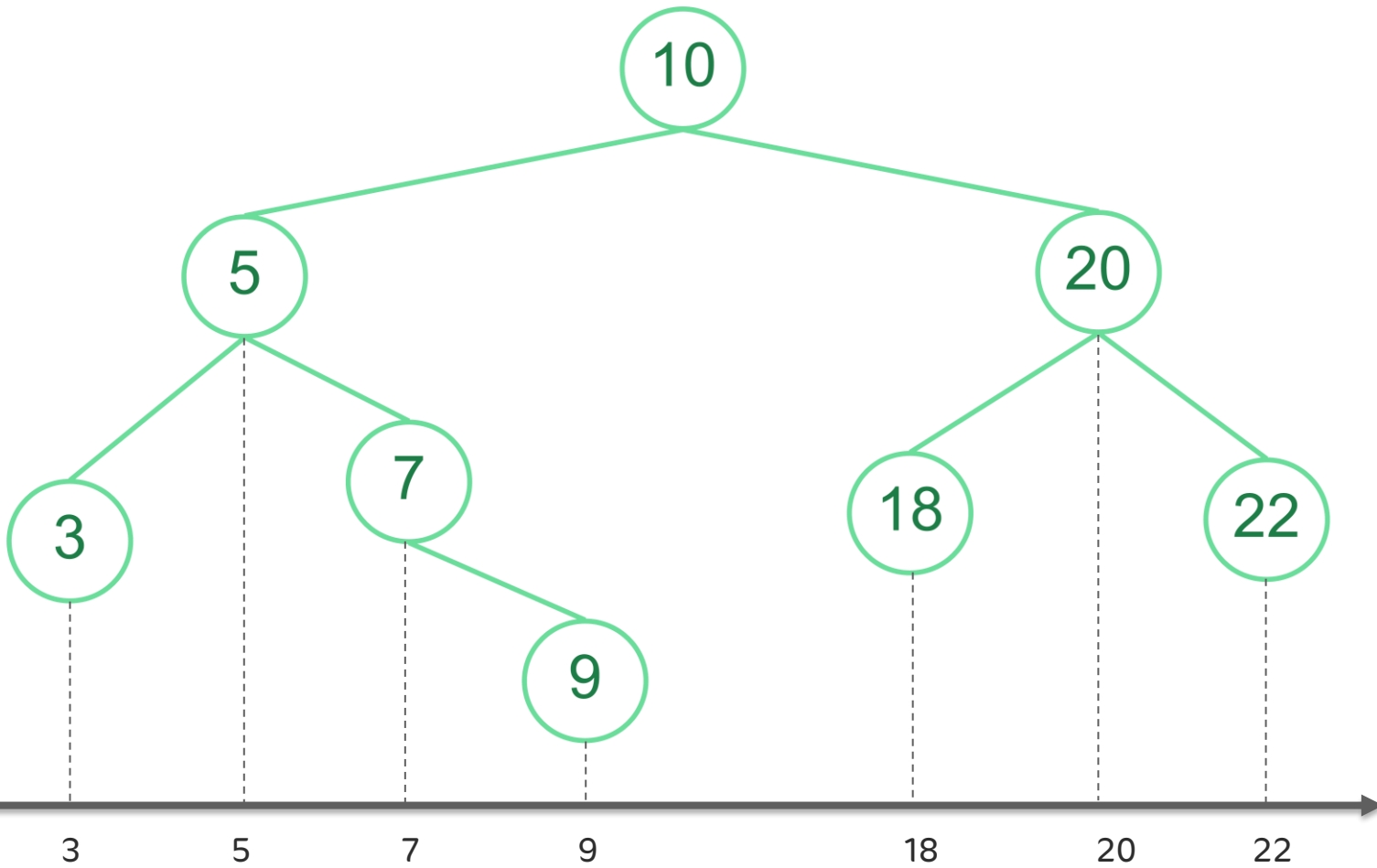
سيكون مكان العنصر
هنا إذا كان تعريف
الشجرة أن الأبناء
اليمين أكبر أو تساوي
الجذر.



العنصر التالي (successor)

نقصد بالعنصر التالي لعقدة معينة بأنه العنصر الذي يليها مباشرة من حيث القيمة، أي لو قمنا بترتيب عناصر الشجرة فما هو العنصر الذي سوف يلي هذه العقدة.

للتوضيح سنقوم بإسقاط عناصر الشجرة التالية على محور الأعداد x :



نلاحظ أن العناصر على المحور تكون مرتبة، أي العنصر الذي يلي (3) هو (5) والعنصر الذي يلي (5) هو (7) والعنصر الذي يلي (20) هو (22).

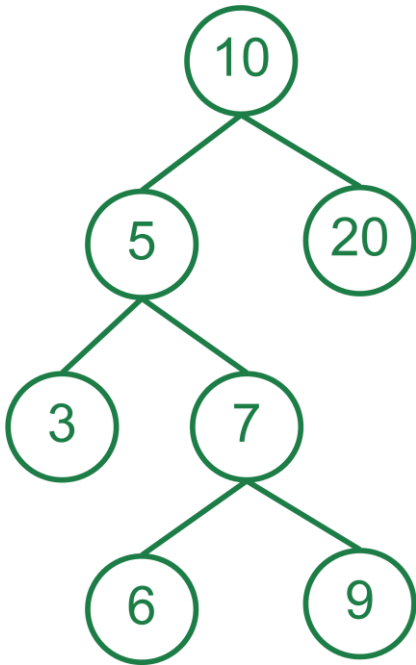


لكن كيف يمكننا التجول في الشجرة لإيجاد العنصر التالي:

يوجد حالتين لإيجاد العنصر التالي (successor):

ليس للعقدة ابن يميني	للعقدة ابن يميني
العنصر التالي هو أول أب أكبر من هذه العقدة، أي نصعد إلى الأعلى حتى نجد أب لهذه العقدة أكبر منها، فيكون هو العنصر المطلوب.	العنصر التالي هو العقدة التي تقع أقصى اليسار في الشجرة الفرعية اليمينية لهذه العقدة. وبمعنى آخر نوجد أصغر قيمة في القسم اليميني (Right subtree) لهذه العقدة، والتي تقع على أقصى اليسار في هذه الشجرة الفرعية.

مثال: ليكن لدينا الشجرة التالية:



- لإيجاد العنصر التالي ل (5): نلاحظ أن للعقدة ابن يميني، فنذهب أولاً إلى اليمين، ثم نستمر في الذهاب إلى اليسار حتى نصل إلى عقدة ليس لها أبناء (حتى نصل إلى الورقة)، أي سوف نصل للعقدة (6). إذاً العنصر التالي ل (5) هو (6).
- لإيجاد العنصر التالي ل (3): نلاحظ أنه ليس للعقدة ابن يميني، لذلك سوف نصعد للأعلى، أي سنصعد للعقدة (5) و $5 > 3$ إذاً العنصر التالي ل (3) هو (5).
- لإيجاد العنصر التالي ل (9): نلاحظ أنه ليس للعقدة ابن يميني، لذلك سوف نصعد للأعلى، أي سنصعد للعقدة (7)، لكن $7 < 9$ وبالتالي سنصعد مجدداً للأعلى للعقدة (5)، ولكن أيضاً $5 < 9$ فنصعد مجدداً إلى ال (10) و $10 > 9$ إذاً العنصر التالي ل (9) هو (10).
- في حالة العنصر (20) نلاحظ أنه لا يوجد آباء أكبر منه لذلك فهو أكبر عنصر ولا يوجد عنصر أكبر منه، أي لا يوجد عنصر تالي لهذه العقدة.

ملاحظة:

إن تعقيد إيجاد العنصر التالي هو من رتبة ارتفاع الشجرة أي التعقيد يساوي $O(H)$.

THE GOAT



الحذف Deletion In BST

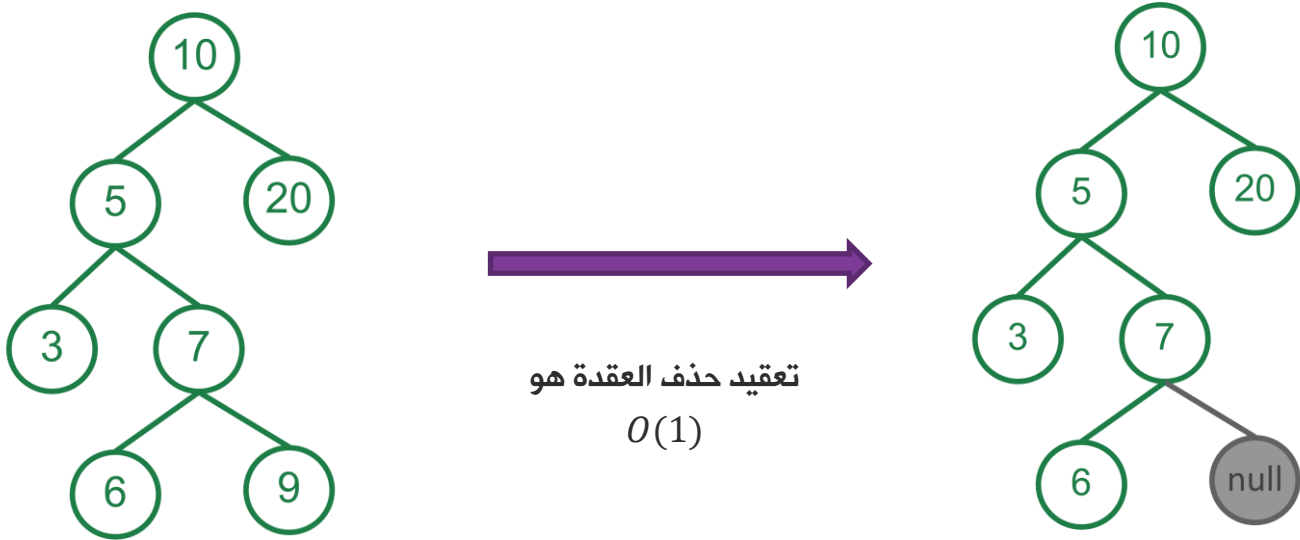
لحذف عقدة من ال BST يوجد لدينا ثلاث حالات:

1. حذف عقدة ليس لها أبناء (حذف ورقة):

إذا كان العنصر الذي نريد حذفه من الشجرة هو ورقة، فنقوم بحذفه مباشرة دون الاضطرار لتعديل الشجرة، أي نجعل العقدة الأب للورقة التي نريد حذفها تُوَشر إلى null.

مثال:

إذا أردنا حذف العقدة (9) من الشجرة التالية، سنجعل مؤشر الابن الأيمن للعقدة (7) يُوَشر إلى null.

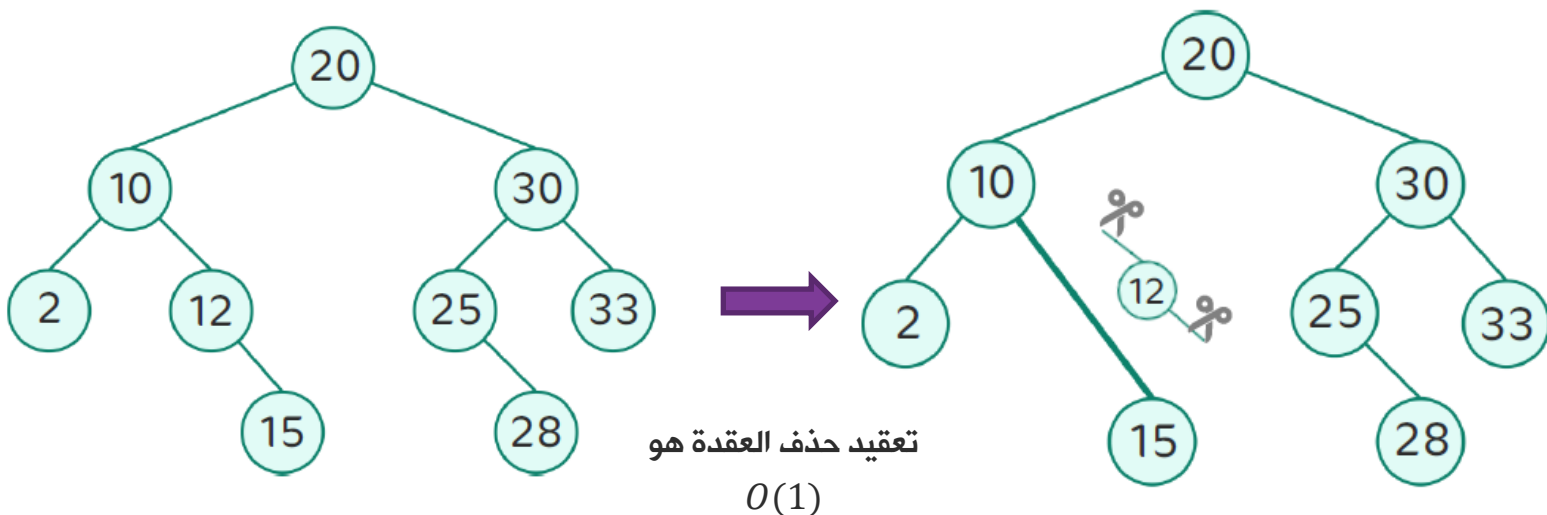


2. العقدة لها ابن وحيد (أيمن أو أيسر):

نحذف العقدة ثم نجعل مؤشر الأب يشير للابن الوحيد لهذه العقدة (قد يكون الابن شجرة فرعية أو ورقة).

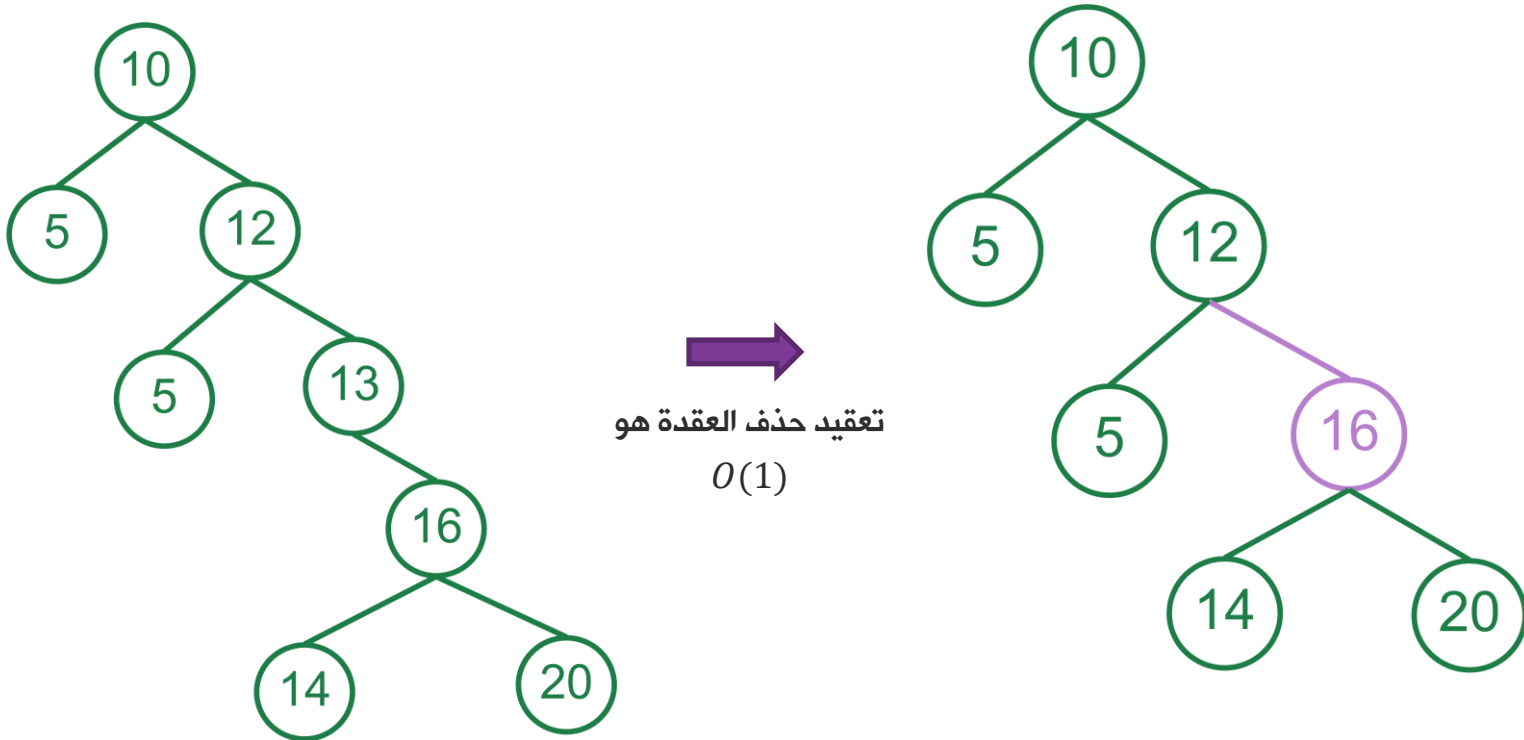
مثال:

إذا أردنا حذف العقدة (12) من الشجرة التالية نجعل المؤشر الأيمن لل 10 (parent) يُوَشر على 15.



مثال:

إذا أردنا حذف العقدة (13) من الشجرة التالية نجعل المؤشر الأيمن لل 12 (parent) يؤول على (16)، وجميع أبناء العقدة (16) سوف ينتقلون معها مهما كان عددهم.



3. حذف عقدة لها ابنان:

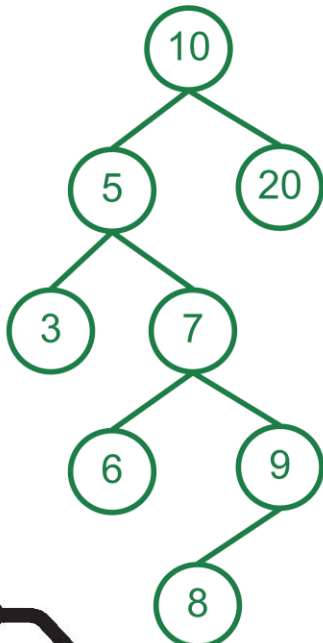
نستبدل العقدة التي نريد حذفها بعنصرها التالي (successor) ثم نحذف العنصر التالي (successor) من مكانه القديم لمنع التكرار.

مثال: إذا أردنا حذف العنصر (7) من الشجرة التالية:

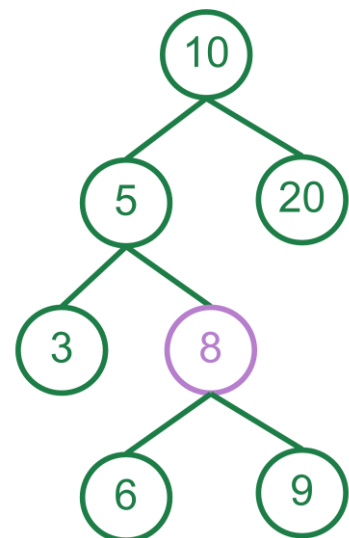
■ نحذف ال (8) من مكانها السابق

■ نستبدل ال (8) مكان ال (7)

■ نوجد العنصر التالي ل (7) وهو (8)



تعقيد حذف العقدة هو $O(H)$



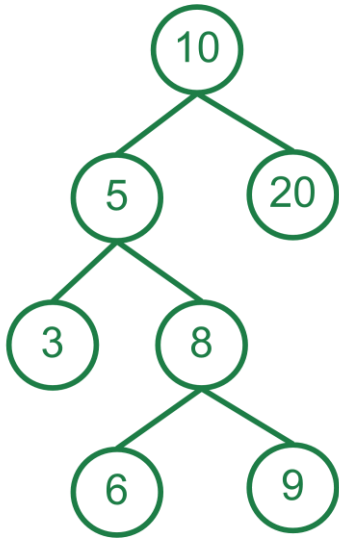
مثال:

إذا أردنا حذف العنصر (5) من الشجرة التالية:

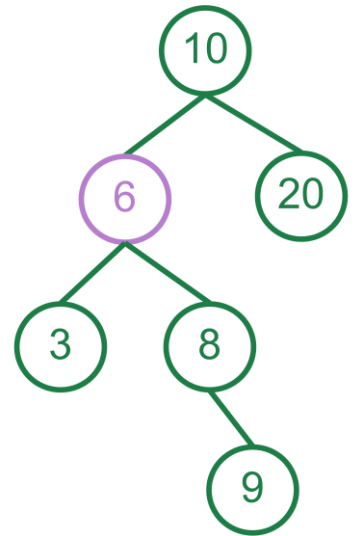
▪ نحذف ال (6) من مكانها السابق

▪ نستبدل ال (6) مكان ال (5)

▪ نوجد العنصر التالي ل (5) وهو (6)



تعقيد حذف العقدة هو $O(H)$



▪ ملاحظة: تعقيد حذف عقدة لها ابنان هو من رتبة ارتفاع الشجرة أي $O(H)$ لأننا نبحث عن العنصر التالي لعقدة ما.

تعقيد الحذف في ال BST بشكل عام:

- أولاً: نبحث عن العنصر بتعقيد $O(H)$.
- ثانياً: نحن أمام حالتين:
 1. إما أن تكون العقدة ورقة أو لها ابن وحيد ← تعقيد الحذف ثابت $O(1)$ لأننا قمنا بتغيير المؤشر فقط.
 2. وإما أن يكون للعقدة ابنان، وفي هذه الحالة نبحث عن العنصر التالي بتعقيد $O(H)$ ، ثم نستبدل العنصر بتعقيد $O(1)$.
- باختصار:

حذف عقدة لها ابنان	حذف عقدة لها ابن وحيد	حذف ورقة
$O(H)$	$O(1)$	$O(1)$

▪ ملاحظة: يمكن استبدال العقدة بالعنصر السابق (predecessor) بدلاً من العنصر التالي (successor). حيث ال العنصر السابق (predecessor) هو أكبر عنصر بالفرع الأيسر للعقدة.

THE END ^_^